

TEAM :- SHORT CIRCUIT SQUAD

TRACK :- IOT

PROBLEM STATEMENT :- HTH-IT-06: Peak-Shaving Energy Control System for Multi-Device Facilities

ESP 32 CODE :-

```
#include <WiFi.h>
```

```
#include <WebServer.h>
```

```
#include <PZEM004Tv30.h>
```

```
// =====
```

```
// WIFI ACCESS POINT
```

```
// =====
```

```
const char* AP_SSID = "ESP32_POWER_MONITOR";
```

```
const char* AP_PASSWORD = "12345678";
```

```
WebServer server(80);
```

```
// =====
```

```
// PZEM-004T V4.0
```

```
// =====
```

```
#define PZEM_RX 32
```

```
#define PZEM_TX 33
```

```
HardwareSerial pzemSerial(2);
```

```

PZEM004Tv30 pzem(
    pzemSerial,
    PZEM_RX,
    PZEM_TX
);

// =====

// RELAYS

// =====

#define RELAY_CRITICAL    25
#define RELAY_NONCRITICAL 26

// Most relay modules are ACTIVE LOW
#define RELAY_ON  LOW
#define RELAY_OFF HIGH

// =====

// CURRENT LIMIT

// =====

float currentLimit = 4.00;

// Relay 2 will restore below this value
float recoveryLimit = 3.50;

// =====

```

```
// MEASUREMENTS
```

```
// =====
```

```
float voltage = 0;
```

```
float current = 0;
```

```
float power = 0;
```

```
float energy = 0;
```

```
float frequency = 0;
```

```
float powerFactor = 0;
```

```
// =====
```

```
// RELAY STATES
```

```
// =====
```

```
bool criticalLoad = true;
```

```
bool nonCriticalLoad = true;
```

```
bool automaticShed = false;
```

```
// =====
```

```
// MANUAL MODE
```

```
// =====
```

```
bool manualMode = false;
```

```
// =====
```

```
// TIMING
```

```
// =====
```

```
unsigned long lastRead = 0;
```

```
const unsigned long READ_INTERVAL = 2000;
```

```
// =====
```

```
// READ PZEM
```

```
// =====
```

```
void readPZEM() {
```

```
    float v = pzem.voltage();
```

```
    float c = pzem.current();
```

```
    float p = pzem.power();
```

```
    float e = pzem.energy();
```

```
    float f = pzem.frequency();
```

```
    float pf = pzem.pf();
```

```
    if (!isnan(v)) {
```

```
        voltage = v;
```

```
    }
```

```
    if (!isnan(c)) {
```

```
        current = c;
```

```
    }
```

```
    if (!isnan(p)) {
```

```
        power = p;
```

```
}
```

```
if (!isnan(e)) {  
    energy = e;  
}
```

```
if (!isnan(f)) {  
    frequency = f;  
}
```

```
if (!isnan(pf)) {  
    powerFactor = pf;  
}
```

```
Serial.println();  
Serial.println("=====");
```

```
Serial.print("Voltage    : ");  
Serial.print(voltage, 2);  
Serial.println(" V");
```

```
Serial.print("Current    : ");  
Serial.print(current, 2);  
Serial.println(" A");
```

```
Serial.print("Power      : ");  
Serial.print(power, 2);  
Serial.println(" W");
```

```
Serial.print("Energy    : ");  
Serial.print(energy, 3);  
Serial.println(" kWh");
```

```
Serial.print("Frequency  : ");  
Serial.print(frequency, 2);  
Serial.println(" Hz");
```

```
Serial.print("Power Factor : ");  
Serial.println(powerFactor, 2);
```

```
Serial.print("Limit      : ");  
Serial.print(currentLimit, 2);  
Serial.println(" A");
```

```
Serial.print("Critical    : ");  
Serial.println(criticalLoad ? "ON" : "OFF");
```

```
Serial.print("Non-Critical : ");  
Serial.println(nonCriticalLoad ? "ON" : "OFF");
```

```
Serial.print("Mode        : ");  
Serial.println>manualMode ? "MANUAL" : "AUTO");
```

```
Serial.print("Status      : ");
```

```
if (automaticShed) {
```

```

        Serial.println("LOAD SHEDDING");
    }
    else {
        Serial.println("NORMAL");
    }

    Serial.println("=====");
}

// =====
// AUTOMATIC LOAD CONTROL
// =====

void automaticLoadControl() {

    if (manualMode) {
        return;
    }

    // -----
    // CURRENT ABOVE LIMIT
    // -----

    if (current > currentLimit) {

        if (nonCriticalLoad) {

            digitalWrite(

```

```
        RELAY_NONCRITICAL,
        RELAY_OFF
    );

    nonCriticalLoad = false;

    automaticShed = true;

    Serial.println();
    Serial.println("!!! CURRENT LIMIT EXCEEDED !!!");
    Serial.println("Non-critical load OFF");
    Serial.println("Critical load remains ON");
}
}

// -----
// CURRENT FALLS BELOW RECOVERY LIMIT
// -----

else if (current < recoveryLimit) {

    if (automaticShed) {

        digitalWrite(
            RELAY_NONCRITICAL,
            RELAY_ON
        );
    }
}
```

```
    nonCriticalLoad = true;

    automaticShed = false;

    Serial.println();
    Serial.println("Current returned to normal");
    Serial.println("Non-critical load ON");
  }
}
}

// =====
// WEB PAGE
// =====

String webpage() {

    String html = R"rawliteral(

<!DOCTYPE html>

<html>

<head>

<meta name="viewport"
content="width=device-width, initial-scale=1">
```

```
<title>ESP32 Smart Power Monitor</title>
```

```
<style>
```

```
body {  
  font-family: Arial;  
  background: #f2f2f2;  
  margin: 0;  
  padding: 15px;  
}
```

```
h1 {  
  text-align: center;  
}
```

```
.card {  
  background: white;  
  padding: 15px;  
  margin: 10px auto;  
  max-width: 500px;  
  border-radius: 12px;  
  box-shadow: 0 2px 8px #aaa;  
}
```

```
.value {  
  font-size: 25px;  
  font-weight: bold;  
}
```

```
.row {  
  display: flex;  
  justify-content: space-between;  
  padding: 8px;  
  border-bottom: 1px solid #ddd;  
}
```

```
button {  
  padding: 12px;  
  margin: 5px;  
  border: none;  
  border-radius: 8px;  
  font-size: 16px;  
}
```

```
.on {  
  background: #4CAF50;  
  color: white;  
}
```

```
.off {  
  background: #f44336;  
  color: white;  
}
```

```
.auto {  
  background: #2196F3;
```

```
    color: white;
}
```

```
input {
    padding: 10px;
    width: 100px;
    font-size: 18px;
}
```

```
.status {
    text-align: center;
    font-size: 22px;
    font-weight: bold;
    padding: 15px;
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h1> ⚡ Smart Power Monitor</h1>
```

```
<div class="card">
```

```
<h2>Electrical Parameters</h2>
```

```
<div class="row">
<span>Voltage</span>
<span class="value" id="voltage">--</span>
</div>
```

```
<div class="row">
<span>Current</span>
<span class="value" id="current">--</span>
</div>
```

```
<div class="row">
<span>Power</span>
<span class="value" id="power">--</span>
</div>
```

```
<div class="row">
<span>Energy</span>
<span class="value" id="energy">--</span>
</div>
```

```
<div class="row">
<span>Frequency</span>
<span class="value" id="frequency">--</span>
</div>
```

```
<div class="row">
<span>Power Factor</span>
<span class="value" id="pf">--</span>
```

```
</div>
```

```
</div>
```

```
<div class="card">
```

```
<h2>Current Limit</h2>
```

```
<center>
```

```
<input
```

```
type="number"
```

```
id="limit"
```

```
step="0.1"
```

```
min="0.1"
```

```
value="4.0">
```

```
<button
```

```
class="auto"
```

```
onclick="setLimit()">
```

```
SET LIMIT
```

```
</button>
```

```
</center>
```

<p>

Recovery level:

3.5 A

</p>

</div>

<div class="card">

<h2>Load Control</h2>

<p>

Critical Load:

<b id="critical">---

</p>

<button

class="on"

onclick="relay1('on')">

Critical ON

</button>

<button

class="off"

onclick="relay1('off')">

Critical OFF

</button>

<hr>

<p>

Non-Critical Load:

<b id="noncritical">--

</p>

<button

class="on"

onclick="relay2('on')">

Non-Critical ON

</button>

<button

class="off"

onclick="relay2('off')">

Non-Critical OFF

</button>

</div>

<div class="card">

<h2>Operating Mode</h2>

<center>

<button

class="auto"

onclick="setAuto()">

AUTO MODE

</button>

<button

class="on"

onclick="setManual()">

MANUAL MODE

</button>

</center>

<p>

Current Mode:

<b id="mode">--

</p>

</div>

<div class="card">

<div

class="status"

id="status">

Loading...

</div>

</div>

<script>

function updateData() {

fetch('/data')

.then(response => response.json())

```
.then(data => {

document.getElementById("voltage").innerHTML =
data.voltage.toFixed(2) + " V";

document.getElementById("current").innerHTML =
data.current.toFixed(2) + " A";

document.getElementById("power").innerHTML =
data.power.toFixed(2) + " W";

document.getElementById("energy").innerHTML =
data.energy.toFixed(3) + " kWh";

document.getElementById("frequency").innerHTML =
data.frequency.toFixed(2) + " Hz";

document.getElementById("pf").innerHTML =
data.pf.toFixed(2);

document.getElementById("limit").value =
data.limit.toFixed(1);

document.getElementById("recovery").innerHTML =
data.recovery.toFixed(1);

document.getElementById("critical").innerHTML =
data.critical ? "ON" : "OFF";
```

```
document.getElementById("noncritical").innerHTML =  
data.noncritical ? "ON" : "OFF";
```

```
document.getElementById("mode").innerHTML =  
data.manual ? "MANUAL" : "AUTO";
```

```
document.getElementById("status").innerHTML =  
data.shed ? " ⚠️ LOAD SHEDDING" : "✓ NORMAL";
```

```
});
```

```
}
```

```
function setLimit() {
```

```
let value =  
document.getElementById("limit").value;
```

```
fetch('/setlimit?value=' + value);
```

```
}
```

```
function relay1(state) {
```

```
fetch('/relay1?state=' + state);
```

```
}
```

```
function relay2(state) {
```

```
  fetch('/relay2?state=' + state);
```

```
}
```

```
function setAuto() {
```

```
  fetch('/auto');
```

```
}
```

```
function setManual() {
```

```
  fetch('/manual');
```

```
}
```

```
setInterval(updateData, 2000);
```

```
updateData();
```

```
</script>
```

```
</body>
```

```
</html>
```

```
)rawliteral";
```

```
    return html;
```

```
}
```

```
// =====
```

```
// WEB ROOT
```

```
// =====
```

```
void handleRoot() {
```

```
    server.send(
```

```
        200,
```

```
        "text/html",
```

```
        webpage()
```

```
    );
```

```
}
```

```
// =====
```

```
// SEND DATA
```

```
// =====
```

```

void handleData() {

    String json = "{";

    json += "\"voltage\":" + String(voltage, 2) + ",";
    json += "\"current\":" + String(current, 2) + ",";
    json += "\"power\":" + String(power, 2) + ",";
    json += "\"energy\":" + String(energy, 3) + ",";
    json += "\"frequency\":" + String(frequency, 2) + ",";
    json += "\"pf\":" + String(powerFactor, 2) + ",";
    json += "\"limit\":" + String(currentLimit, 2) + ",";
    json += "\"recovery\":" + String(recoveryLimit, 2) + ",";
    json += "\"critical\":" + String(criticalLoad ? "true" : "false") + ",";
    json += "\"noncritical\":" + String(nonCriticalLoad ? "true" : "false") + ",";
    json += "\"manual\":" + String>manualMode ? "true" : "false") + ",";
    json += "\"shed\":" + String(automaticShed ? "true" : "false");

    json += "}";

    server.send(
        200,
        "application/json",
        json
    );
}

// =====

```

```
// SET CURRENT LIMIT
// =====

void handleSetLimit() {

    if (server.hasArg("value")) {

        float newLimit =
            server.arg("value").toFloat();

        if (newLimit > 0.1 && newLimit <= 100.0) {

            currentLimit = newLimit;

            recoveryLimit =
                currentLimit - 0.5;

            if (recoveryLimit < 0.1) {
                recoveryLimit = 0.1;
            }

            Serial.print("New current limit: ");
            Serial.print(currentLimit);
            Serial.println(" A");

            Serial.print("Recovery limit: ");
            Serial.print(recoveryLimit);
            Serial.println(" A");
```

```

    }
}

server.send(
    200,
    "text/plain",
    "OK"
);
}

// =====
// CRITICAL RELAY
// =====

void handleRelay1() {

    if (server.hasArg("state")) {

        String state =
            server.arg("state");

        if (state == "on") {

            digitalWrite(
                RELAY_CRITICAL,
                RELAY_ON
            );

```

```
        criticalLoad = true;
    }

    else if (state == "off") {

        digitalWrite(
            RELAY_CRITICAL,
            RELAY_OFF
        );

        criticalLoad = false;
    }

}

server.send(
    200,
    "text/plain",
    "OK"
);
}

// =====
// NON-CRITICAL RELAY
// =====

void handleRelay2() {
```

```
if (server.hasArg("state")) {

    String state =
        server.arg("state");

    if (state == "on") {

        digitalWrite(
            RELAY_NONCRITICAL,
            RELAY_ON
        );

        nonCriticalLoad = true;

        automaticShed = false;
    }

    else if (state == "off") {

        digitalWrite(
            RELAY_NONCRITICAL,
            RELAY_OFF
        );

        nonCriticalLoad = false;
    }

}
```

```
server.send(
  200,
  "text/plain",
  "OK"
);
}

// =====
// AUTO MODE
// =====

void handleAuto() {

  manualMode = false;

  Serial.println("AUTO MODE");

  server.send(
    200,
    "text/plain",
    "AUTO"
  );
}

// =====
// MANUAL MODE
// =====
```

```
void handleManual() {

    manualMode = true;

    automaticShed = false;

    Serial.println("MANUAL MODE");

    server.send(
        200,
        "text/plain",
        "MANUAL"
    );
}

// =====
// SETUP
// =====

void setup() {

    Serial.begin(115200);

    delay(1000);

    Serial.println();
    Serial.println("=====");
```

```
Serial.println("ESP32 SMART POWER MONITOR");  
Serial.println("=====");
```

```
// -----  
// RELAY SETUP  
// -----
```

```
pinMode(  
    RELAY_CRITICAL,  
    OUTPUT  
);
```

```
pinMode(  
    RELAY_NONCRITICAL,  
    OUTPUT  
);
```

```
// Both loads ON at startup  
digitalWrite(  
    RELAY_CRITICAL,  
    RELAY_ON  
);
```

```
digitalWrite(  
    RELAY_NONCRITICAL,  
    RELAY_ON  
);
```

```
criticalLoad = true;
```

```
nonCriticalLoad = true;
```

```
// -----
```

```
// PZEM
```

```
// -----
```

```
pzemSerial.begin(
```

```
  9600,
```

```
  SERIAL_8N1,
```

```
  PZEM_RX,
```

```
  PZEM_TX
```

```
);
```

```
delay(1000);
```

```
// -----
```

```
// WIFI ACCESS POINT
```

```
// -----
```

```
WiFi.mode(WIFI_AP);
```

```
WiFi.softAP(
```

```
  AP_SSID,
```

```
  AP_PASSWORD
```

```
);
```

```
IPAddress IP =
```

```
WiFi.softAPIP();
```

```
Serial.println();
```

```
Serial.println("WiFi Access Point Started");
```

```
Serial.print("SSID: ");
```

```
Serial.println(AP_SSID);
```

```
Serial.print("Password: ");
```

```
Serial.println(AP_PASSWORD);
```

```
Serial.print("Dashboard IP: ");
```

```
Serial.println(IP);
```

```
// -----
```

```
// WEB SERVER
```

```
// -----
```

```
server.on(
```

```
    "/",
```

```
    handleRoot
```

```
);
```

```
server.on(
```

```
    "/data",
```

```
    handleData
```

```
);
```

```
server.on(  
  "/setlimit",  
  handleSetLimit  
);
```

```
server.on(  
  "/relay1",  
  handleRelay1  
);
```

```
server.on(  
  "/relay2",  
  handleRelay2  
);
```

```
server.on(  
  "/auto",  
  handleAuto  
);
```

```
server.on(  
  "/manual",  
  handleManual  
);
```

```
server.begin();
```

```
Serial.println();
```

```
Serial.println("Web server started");
```

```
Serial.println();
```

```
Serial.println("Connect your phone/PC to:");
```

```
Serial.println(AP_SSID);
```

```
Serial.println();
```

```
Serial.println("Then open:");
```

```
Serial.println("http://192.168.4.1");
```

```
}
```

```
// =====
```

```
// LOOP
```

```
// =====
```

```
void loop() {
```

```
    server.handleClient();
```

```
    if (
```

```
        millis() - lastRead >= READ_INTERVAL
```

```
    ) {
```

```
        lastRead = millis();
```

```
        readPZEM();
```

```
        automaticLoadControl();
```

}

}